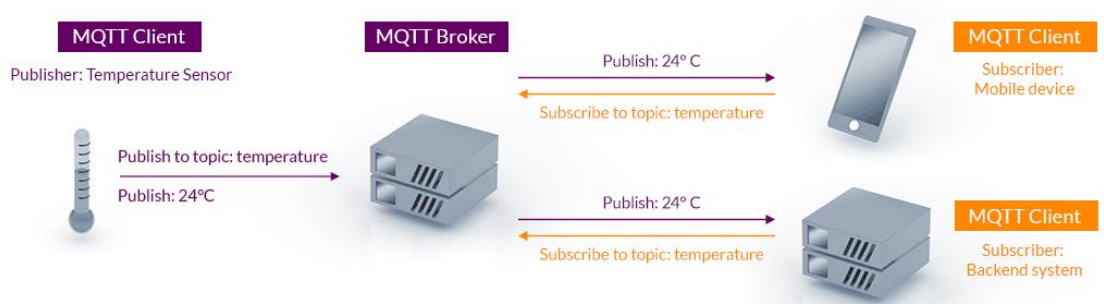
ΤΙΤΛΟΣ: ΘΩΡΑΚΙΣΗ (HARDENING) ΔΙΑΚΟΜΙΣΤΗ MQTT ΕΝΟΣ ΣΥΣΤΗΜΑΤΟΣ IOT

Σε ένα σύστημα Internet of Things (IoT), η δομή του MQTT Broker (διακομιστή) αποτελεί τον ακρογωνιαίο λίθο για την μεταφορά των δεδομένων από τους αισθητήρες στα εποπτικά συστήματα και στις πλατφόρμες IoT. Ο MQTT Broker είναι ο κεντρικός διαμεσολαβητής σε ένα σύστημα MQTT (Message Queuing Telemetry Transport) που λειτουργεί ως server για τη διαχείριση και διανομή μηνυμάτων. Το σύστημα MQTT (https://mqtt.org/) αποτελεί ένα ελαφρύ πρωτόκολλο ανταλλαγής μηνυμάτων σχεδιασμένο για δίκτυα χαμηλού εύρους ζώνης, μικρού όμως TCP header σε σχέση με τη κλασική υποδομή TCP/IP, δηλαδή εκτελείται μέσω ενός πρωτοκόλλου μεταφοράς που παρέχει διατεταγμένες, χωρίς απώλειες, αμφίδρομες συνδέσεις TCP/IP. Αποτελεί ένα ανοιχτό πρότυπο OASIS και μια σύσταση του ISO (ISO/IEC 20922).

Η αρχιτεκτονική του βασίζεται στη δομή Publish/Subscribe, σύμφωνα με την οποία, ο MQTT Broker παραλαμβάνει μηνύματα από publishers (αποστολείς) βάσει συγκεκριμένων θεμάτων (topics) και τα προωθεί στους subscribers (παραλήπτες), οι οποίοι έχουν εγγραφεί σε αυτά τα θέματα. Υποστηρίζει πολλαπλές συνδέσεις ταυτόχρονα, εξασφαλίζοντας αποδοτική επικοινωνία σε δίκτυα IoT με περιορισμένους πόρους. Όπως γίνεται αντιληπτό, σύμφωνα με το μοντέλο επικοινωνίας publish/subscribe, οι συσκευές δεν επικοινωνούν απευθείας, αλλά μέσω του broker, που διαχειρίζεται τις συνδέσεις, την ασφάλεια (π.χ. SSL/TLS) και την ποιότητα της υπηρεσίας (QoS). Στο παρακάτω σχήμα δίνεται ένα παράδειγμα τέτοιας αρχιτεκτονικής βάσει του μοντέλου αυτού.



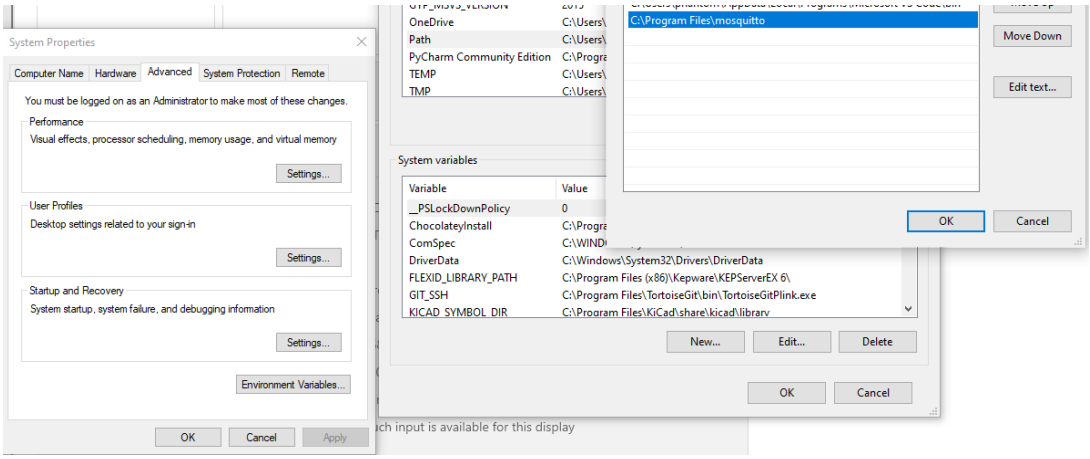
Εικόνα 1. Αρχιτεκτονική Publish Subscribe του Πρωτοκόλλου Επικοινωνίας MQTT (πηγή: https://mqtt.org/)

Αντικειμενικοί στόχοι της εργασίας αποτελούν: α) η εγκατάσταση του MQTT broker και η παραμετροποίησή του, β) η κατανόηση του μοντέλου publish/subscribe και της αρχής λειτουργίας τους και γ) η θωράκιση του MQTT broker από κυβερνοεπιθέσεις μιας και αποτελεί το κρίσιμο και απαραίτητο δομικό σημείο επικοινωνίας για τη λειτουργία ενός συστήματος IoT.

Εγκατάσταση και Έλεγχος Mosquitto MQTT Broker σε Windows

Για την εγκατάσταση του MQTT broker **Eclipse Mosquitto** σε περιβάλλον Windows, μεταβαίνετε στη σελίδα λήψεων του Mosquitto στη διεύθυνση https://mosquitto.org/download/. Στην ενότητα **Windows** κατεβάζετε το αρχείο εγκατάστασης **mosquitto-2.1.2-install-windows-x64.exe**.

Στη συνέχεια, εκτελείτε το αρχείο εγκατάστασης **ως Διαχειριστής**. Κατά τη διάρκεια της εγκατάστασης είναι σημαντικό να επιλεγούν οι επιλογές **Install Service**, ώστε ο Mosquitto Broker να εγκατασταθεί ως Windows Service και να ξεκινά αυτόματα, καθώς και **Add mosquitto to PATH**, ώστε να είναι διαθέσιμες οι εντολές mosquitto, mosquitto_pub και mosquitto_sub από τη γραμμή εντολών (**αν στις Μεταβλητές Περιβάλλοντος στο PATH δεν υφίσταται το** C:\Program Files\mosquitto, το προσθέτετε).



Μετά την ολοκλήρωση της εγκατάστασης, ο broker είναι έτοιμος για χρήση. Μπορείτε να ελέγξετε ότι ο Mosquitto broker εκτελείται ως υπηρεσία των Windows. Η προτεινόμενη μέθοδος είναι μέσω του γραφικού περιβάλλοντος: πιέστε **Win + R**, πληκτρολογείτε **services.msc** και εντοπίζετε την υπηρεσία **Mosquitto Broker**, η οποία θα πρέπει να βρίσκεται σε κατάσταση **Running (Αν δεν είναι την ενεργοποιείτε)**. Εναλλακτικά, από τη γραμμή εντολών χρησιμοποιήστε την εντολή **sc query mosquitto**.

```
SERVICE_NAME: mosquitto
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

Για βασική δοκιμή λειτουργίας του broker, ανοίγετε δύο παράθυρα **Command Prompt**. Στο πρώτο εκτελέστε έναν subscriber με την εντολή:

mosquitto_sub -h localhost -t test/topic

και στο δεύτερο έναν publisher:

mosquitto_pub -h localhost -t test/topic -m "Hello MQTT!"

Αν το μήνυμα εμφανιστεί στο παράθυρο του subscriber, τότε ο Mosquitto broker λειτουργεί σωστά.

Αν όχι, τότε στο αρχείο ρυθμίσεων του Mosquitto **mosquitto.conf** που βρίσκεται στον φάκελο: **C:\Program Files\mosquitto** το οποίο το ανοίγετε ως Διαχειριστής από notepad να προσθέσετε τις ρυθμίσεις:

listener 1883
allow_anonymous true

δείτε παρακάτω τμήμα το αρχείου με τις εντολές αυτές.

```
# listener port-number [ip address/host name/unix socket path]
listener 1883
allow_anonymous true
```

Οι ρυθμίσεις αυτές επιτρέπουν ανώνυμη πρόσβαση και προορίζονται **μόνο για δοκιμές ή τοπικό δίκτυο στο port 1883**. Μετά από οποιαδήποτε αλλαγή στο αρχείο ρυθμίσεων απαιτείται επανεκκίνηση του broker **με τις εντολές από παράθυρο CMD το οποίο το έχετε ανοίξει ως διαχειριστής**:

net stop mosquitto
net start mosquitto

και **sc query mosquitto** για να ελέγξετε αν είναι σε κατάσταση **RUNNING**
Επίσης με την εντολή netstat -ano | findstr :1883

```
TCP    0.0.0.0:1883           0.0.0.0:0              LISTENING       15388
TCP    [::]:1883              [::]:0                 LISTENING       15388
```

Αν παρουσιαστούν περαιτέρω προβλήματα σύνδεσης από clients, ενδέχεται να απαιτείται ρύθμιση του **Windows Defender Firewall** ώστε να επιτραπεί η εισερχόμενη και εξερχόμενη επικοινωνία στην πόρτα **1883 (TCP)**.

Για την υλοποίηση της εργασίας, σας δίνονται τα παρακάτω python script, τα οποία θα τα αντιγράψετε στα αντίστοιχα αρχεία σας (publisher.py & subscriber.py). Θα πρέπει να

εγκαταστήσετε στη python που έχετε στον υπολογιστή σας τη βιβλιοθήκη paho-mqtt **(σε παράθυρο cmd εκτελέστε pip install paho-mqtt**). Αν δεν έχετε εγκαταστημένο το pip τότε εκτελέστε **python.exe -m pip install --upgrade pip** σε CMD και στη συνέχεια **pip install paho-mqtt**.

Αρχείο 1: publisher.py

```
import paho.mqtt.client as mqtt
import json
import time
import random
import uuid
from datetime import datetime, UTC

BROKER = "localhost"
PORT = 1883
TOPIC = "iot/temperature"

CLIENT_ID = f"iot-publisher-{uuid.uuid4().hex[:8]}"

def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print(f"Publisher connected with CLIENT_ID: {CLIENT_ID}")
    else:
        print(f"Failed to connect, return code {reason_code}")

client = mqtt.Client(
    client_id=CLIENT_ID,
    callback_api_version=mqtt.CallbackAPIVersion.VERSION2
)

client.on_connect = on_connect

client.connect(BROKER, PORT, keepalive=60)
client.loop_start()

try:
    while True:
        payload = {
            "sensor_id": "temp_sensor",
            "temperature_c": round(random.uniform(20.0, 30.0), 2),
            "timestamp": datetime.now(UTC).isoformat()
        }
```

```
        client.publish(TOPIC, json.dumps(payload))
        print(f"Published: {payload}")

        time.sleep(5)

except KeyboardInterrupt:
    print("Stop publisher!")
    client.loop_stop()
    client.disconnect()
```

Αρχείο 2: subscriber.py

```
import paho.mqtt.client as mqtt
import random
import string

def generate_client_id(prefix):
    random_suffix = ''.join(random.choices(string.ascii_lowercase + string.digits, k=6))
    return f"{prefix}-{random_suffix}"

broker = "localhost"
port = 1883
topic = "iot/temperature"

client_id = generate_client_id("subscriber")

print(f"Subscriber ON - Client ID: {client_id}")

def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print("Connected to MQTT broker successfully")
        client.subscribe(topic, qos=0)
    else:
        print(f"Connection failed with code {reason_code}")

def on_message(client, userdata, msg):
    print(f"Received on {msg.topic}: {msg.payload.decode()}")

client = mqtt.Client(
    client_id=client_id,
    callback_api_version=mqtt.CallbackAPIVersion.VERSION2
)
```

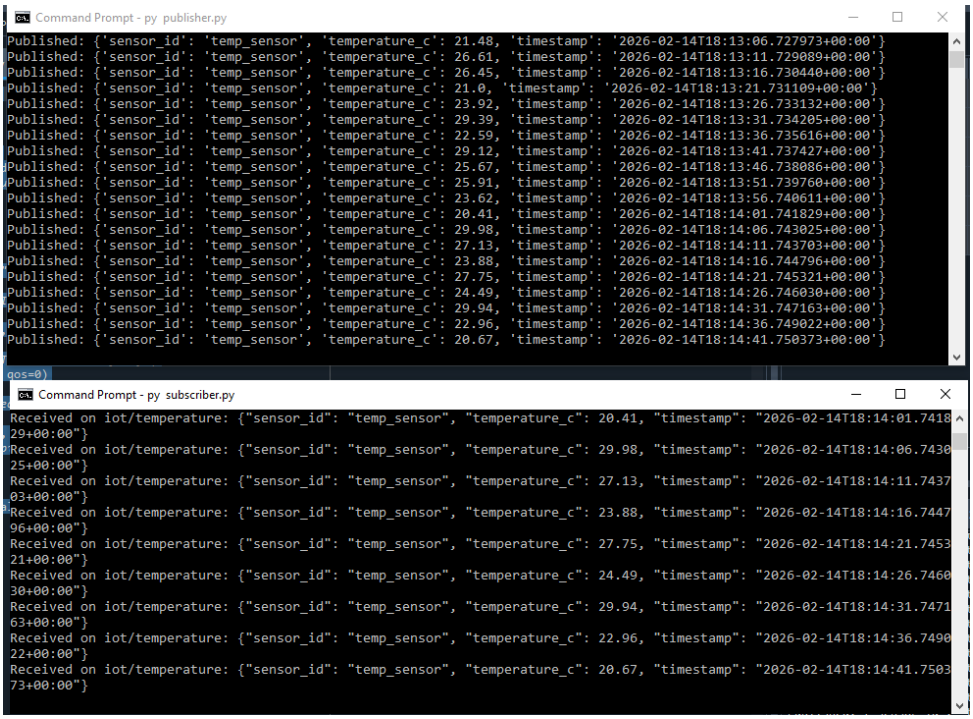
```
client.on_connect = on_connect
client.on_message = on_message

client.connect(broker, port)
client.loop_forever()
```

Χρήση των αρχείων publisher.py και subscriber.py με τον Mosquitto

Αφού ο Mosquitto broker τρέχει τοπικά, τα αρχικά Python scripts **publisher.py** και **subscriber.py** μπορούν να χρησιμοποιηθούν **χωρίς καμία αλλαγή στη λογική τους** και να εκτελέσετε τα 2 αρχεία python ταυτόχρονα σε διαφορετικό παράθυρο εκτέλεσης π.χ. 2 CMD console για να επαληθεύσετε τη λειτουργία του broker και των 2 client. Έστω ότι έχετε αποθηκεύσει τα αρχεία publisher.py και subscriber.py στο C:\project\.

Τότε στο 1ο CMD console θα εκτελέσετε:
cd C:\project
py subscriber.py
Στο 2ο CMD console θα εκτελέσετε:
cd C:\project
py publisher.py στη συνέχεια θα δείτε ότι μέσω του MQTT Broker του υπολογιστή σας θα λαμβάνετε τα μηνύματα μεταξύ των 2 CMD console.



Ο **publisher.py** λειτουργεί ως client που δημοσιεύει δεδομένα (π.χ. μετρήσεις αισθητήρα θερμοκρασίας) σε ένα συγκεκριμένο topic το **iot/temperature**, ενώ ο **subscriber.py** εγγράφεται στο ίδιο topic και λαμβάνει τα μηνύματα. Εκτελώντας πρώτα τον subscriber και στη συνέχεια τον publisher από διαφορετικά Command Prompts, επιβεβαιώνεται η σωστή λειτουργία του broker και η ανταλλαγή μηνυμάτων μέσω MQTT.

Στόχος της άσκησης είναι η **σταδιακή θωράκιση** ενός Mosquitto broker μέσω παραμετροποίησης του αρχείου mosquitto.conf, καθώς και η ανάλυση ζητημάτων **διαθεσιμότητας και ανθεκτικότητας**.

Ο MQTT broker **Eclipse Mosquitto** αποτελεί βασικό στοιχείο σε IoT και distributed συστήματα. Από προεπιλογή, η εγκατάστασή του **δεν είναι ασφαλής**, καθώς επιτρέπει ανώνυμη πρόσβαση και μη κρυπτογραφημένη επικοινωνία.

ΠΡΟΣΟΧΗ: ΠΡΙΝ ΟΠΟΙΑΔΗΠΟΤΕ ΕΝΕΡΓΕΙΑ ΠΡΟΣ ΤΗΝ ΕΠΙΛΥΣΗ ΤΑ ΠΑΡΑΚΑΤΩ ΕΡΩΤΗΜΑΤΑ, ΝΑ ΑΝΤΙΓΡΑΨΕΤΕ ΤΟ ΑΡΧΙΚΟ ΑΡΧΕΙΟ mosquitto.conf ΣΕ ΑΛΛΟ ΦΑΚΕΛΟ ΤΗΣ ΕΠΙΛΟΓΗΣ ΓΙΑ BACKUP, ΔΙΟΤΙ ΘΑ ΑΠΑΙΤΗΘΕΙ ΝΑ ΠΡΑΓΜΑΤΟΠΟΙΗΣΕΤΕ ΑΛΛΑΓΕΣ.

ΣΕ ΠΕΡΙΠΤΩΣΗ ΠΟΥ ΜΕΤΑ ΤΙΣ ΑΛΛΑΓΕΣ ΣΤΟ ΑΡΧΕΙΟ mosquitto.conf ΝΑ ΑΝΟΙΞΕΤΕ ΕΝΑ ΝΕΟ ΠΑΡΑΘΥΡΟ CMD ΚΑΙ ΝΑ ΕΚΤΕΛΕΣΕΤΕ cd C:\Program Files\mosquitto και στη συνέχεια mosquitto.exe -c mosquitto.conf –v και στη συνέχεια mosquitto.exe -c mosquitto.conf -v

ΕΡΩΤΗΜΑΤΑ:

Ερώτημα 1 – Αυθεντικοποίηση Clients με Username / Password

Να ρυθμιστεί ο Mosquitto broker ώστε να επιτρέπει σύνδεση **μόνο σε πιστοποιημένους χρήστες**, μέσω username και password. Ο broker δεν θα πρέπει να δέχεται clients χωρίς credentials.

- Χρησιμοποίηστε το πρόγραμμα mosquitto_passwd.exe που βρίσκεται στο C:\Program Files\mosquitto\ για τη δημιουργία των στοιχείων πρόσβασης
- Στο mosquitto.conf θα πρέπει να δώσετε τις σωστές παραμέτρους (password_file κτλ)
- **Προσοχή:** Οι αλλαγές απαιτούν **επανεκκίνηση** του broker αφού πρώτα **ανοίξετε ένα παράθυρο Command Prompt** ως **Διαχειριστής** και εκτελέσετε είτε net stop mosquitto και στη συνέχεια **net start mosquitto** ή **sc stop mosquitto** και στη συνέχεια **sc start mosquitto**, αναλόγως με την εγκατάστασή σας. Σε CMD console εκτελέστε **netstat -ano | findstr :1883 για να βεβαιωθείτε αν είναι ON ο broker, αν όχι εκτελέστε cd C:\Program Files\mosquitto και στη συνέχεια mosquitto.exe -c mosquitto.conf –v**
- Θα πρέπει να προσαρμόσετε τον κώδικα python του publisher και του subscriber αναλόγως και το mosquitto.conf, όπως αναφέρθηκε στην αρχή της εκφώνησης.

Ερώτημα 2 – Έλεγχος Πρόσβασης με ACL (Access Control Lists)

Σε συνέχεια του ερωτήματος 1, να υλοποιηθεί μηχανισμός **ελέγχου πρόσβασης σε topics** χρησιμοποιώντας **ACLs** στον Mosquitto broker. Θα πρέπει να παραμετροποιήσετε το αρχείο ρυθμίσεων: C:\Program Files\mosquitto\ mosquitto.conf. Ο MQTT broker θα πρέπει:

- να **απαγορεύει ανώνυμη πρόσβαση**
- να επιτρέπει σε διαφορετικούς clients:
    - publish μόνο σε συγκεκριμένα topics (**θα πρέπει να θέσετε το δικό σας topic** το οποίο θα δέχεται την αποστολή και λήψη δεδομένων)
    - subscribe μόνο σε συγκεκριμένα topics (**θα πρέπει να θέσετε το δικό σας topic** το οποίο θα δέχεται την αποστολή και λήψη δεδομένων))
    - Να ελέγξετε αν μετά την εφαρμογή του ACL έχετε αποτρέψει τον publisher ή τον subscriber να στέλνει δεδομένα σε topic διαφορετικά από το white list του ACL που έχετε εφαρμόσει.
- **Προσοχή:** Οι αλλαγές απαιτούν **επανεκκίνηση** του broker αφού πρώτα **ανοίξετε ένα παράθυρο Command Prompt** ως **Διαχειριστής** και εκτελέσετε είτε net stop mosquitto και στη συνέχεια **net start mosquitto** ή **sc stop mosquitto** και στη συνέχεια **sc start mosquitto**, αναλόγως με την εγκατάστασή σας. Σε CMD console εκτελέστε **netstat -ano | findstr :1883 για να βεβαιωθείτε αν είναι ON ο broker, αν όχι εκτελέστε cd C:\Program Files\mosquitto και στη συνέχεια mosquitto.exe -c mosquitto.conf –v**
- Θα πρέπει να προσαρμόσετε τον κώδικα python του publisher και του subscriber αναλόγως και το mosquitto.conf, όπως αναφέρθηκε στην αρχή της εκφώνησης.

Ερώτημα 3 – Ασφαλής Επικοινωνία με TLS / Certificates

Να υλοποιηθεί **συμμετρική κρυπτογραφημένη επικοινωνία (SSL/TLS)** μεταξύ MQTT clients και Mosquitto broker με χρήση **πιστοποιητικών (certificates)**. Θα πρέπει να παραμετροποιήσετε το αρχείο ρυθμίσεων: C:\Program Files\mosquitto\ mosquitto.conf ενώ το Certificate θα εκδοθεί με εγκατάσταση και χρήση του **OpenSSL** (εγκατάσταση σε **περιβάλλον Windows για πληροφορίες για το εκτελέσιμο Win64OpenSSL_Light-3_6_1.msi επισκεφτείτε τον ιστότοπο το σύνδεσμο** https://slproweb.com/products/Win32OpenSSL.html). Το πρόγραμμα θα εγκατασταθεί στο C:\Program Files\OpenSSL-Win64. Μπορείτε να χρησιμοποιήσετε τον αλγόριθμο genrsa με αριθμό bits της επιλογής σας. Το Κοινό Όνομα (CN) πρέπει να ταιριάζει με το όνομα του mqtt broker σας κατά την έκδοση των πιστοποιητικών. Επομένως, να να δώσετε το **localhost**.

Hints:

- Ο Mosquitto υποστηρίζει TLS μέσω:
    - cafile
    - certfile
    - keyfile
    - listener 8883

- **Προσοχή:** Οι αλλαγές απαιτούν **επανεκκίνηση** του broker αφού πρώτα **ανοίξετε ένα παράθυρο Command Prompt** ως **Διαχειριστής** και εκτελέσετε είτε net stop mosquitto

και στη συνέχεια **net start mosquitto** ή **sc stop mosquitto** και στη συνέχεια **sc start mosquitto**, αναλόγως με την εγκατάστασή σας. Σε CMD console εκτελέστε **netstat -ano | findstr :1883 για να βεβαιωθείτε αν είναι ON ο broker, αν όχι εκτελέστε cd C:\Program Files\mosquitto και στη συνέχεια mosquitto.exe -c mosquitto.conf –v**

- Θα πρέπει να προσαρμόσετε τον κώδικα python του publisher και του subscriber αναλόγως και το mosquitto.conf, όπως αναφέρθηκε στην αρχή της εκφώνησης.

Ερώτημα 4 – Single Point of Failure & Αντιμετώπιση DDoS

Ένας MQTT broker εγκατεστημένος σε **έναν μόνο server** αποτελεί **Single Point of Failure (SPOF)** και είναι ευάλωτος σε επιθέσεις τύπου **DDoS**.

Να προτείνετε αρχιτεκτονικές και τεχνικές λύσεις που:

i. Αυξάνουν τη **διαθεσιμότητα** του συστήματος

ii. Περιορίζουν ή μετριάζουν επιθέσεις DDoS

ΠΑΡΑΔΟΤΕΟ: Ένα αρχείο pdf ή word με απαντήσεις για το κάθε ερώτημα. Για το κάθε ένα από τα ερωτήματα 1, 2 και 3 να παραθέσετε τους κώδικες των publisher.py & subscriber.py καθώς και το configuration file mosquitto.conf μαζί με τα βήματα που έχετε ακολουθήσει για την υλοποίησή σας.